Лекция 4. Системы линейных алгебраических уравнений

Вычислительная математика,

Весенний семестр 2024

Перл Ольга Вячеславовна

План лекции

- Простые методы решения СЛАУ с 2 неизвестными
- Прямые методы
- Итерационные методы

Общие сведения о системах уравнений

- Системой n уравнений с m неизвестных $x_1, x_2, \dots, x_m$ принадлежащими заданному числовому множеству M называется задача о нахождении упорядоченных совокупностей из m чисел $(\alpha_1, \alpha_2, \dots, \alpha_m)$ принадлежащих множеству M, которые при подстановке вместо соответствующих неизвестных обращают каждое из n данных уравнений в тождество.

Общий вид системы уравнений

- $$\begin{cases} F_1(x_1, x_2, \dots, x_m) = \Phi_1(x_1, x_2, \dots, x_m) \\ F_2(x_1, x_2, \dots, x_m) = \Phi_2(x_1, x_2, \dots, x_m) \\ \dots \\ F_n(x_1, x_2, \dots, x_m) = \Phi_n(x_1, x_2, \dots, x_m) \end{cases} \quad (1)$$

Решение системы уравнений

- Упорядоченная совокупность чисел $\alpha_1, \alpha_2, \dots, \alpha_m$, каждое из которых принадлежит множеству M, в котором рассматривается система, при подстановке которых в место соответствующих неизвестных каждое уравнение системы обращается в тождество.

Решить систему уравнений

- - Означает найти множество всех её решений или показать, что она решений не имеет.

Решение систем линейных уравнений с 2 неизвестными

- Общий вид СЛАУ с двумя неизвестными:

$$\begin{cases} a_1 x + b_1 y = c_1, \\ a_2 x + b_2 y = c_2, \end{cases} \quad (2)$$

где x и y – неизвестные; а a_1, a_2, b_1, b_2 - коэффициенты; c_1, c_2 - столбец свободных членов (свободными членами системы).

- Методы решения СЛАУ с двумя неизвестными:

1. Метод подстановки
2. Метод алгебраического сложения
3. Метод сравнения

Метод подстановки

- Из (2) при $b_1 \neq 0$ мы находим:

$$y = \frac{c_1 - a_1 x}{b_1}$$

- И подставляем найденное значение y во второе уравнение:

$$a_2 x + b_2 \frac{c_1 - a_1 x}{b_1} = c_2$$

- Откуда получаем:

$$(a_1 b_2 - a_2 b_1) x = c_1 b_2 - c_2 b_1$$

- Если $(a_1 b_2 - a_2 b_1) \neq 0$, то:

$$x = \frac{c_1 b_2 - c_2 b_1}{a_1 b_2 - a_2 b_1}$$

- Тогда:

$$y = \frac{c_1}{b_1} - \frac{a_1}{b_1} x = \frac{c_1}{b_1} - \frac{a_1 (c_1 b_2 - c_2 b_1)}{b_1 (a_1 b_2 - a_2 b_1)} = \frac{a_1 c_2 - a_2 c_1}{a_1 b_2 - a_2 b_1}$$

Метод алгебраического сложения

- Для решения системы (2), умножим обе части первого уравнения $b_2 \neq 0$, а второго – на $-b_1 \neq 0$:

$$\begin{cases} a_1 b_2 x + b_1 b_2 y = c_1 b_2, \\ -a_2 b_1 x + b_1 b_2 y = -c_2 b_1, \end{cases}$$

- Полагая, что система имеет решение, складываем левые и правые части уравнений:

$$(a_1 b_2 - a_2 b_1)x = c_1 b_2 - c_2 b_1,$$

- Откуда при $a_1 b_2 - a_2 b_1 \neq 0$ находим:

$$x = \frac{c_1 b_2 - c_2 b_1}{a_1 b_2 - a_2 b_1}$$

- Аналогично поступаем, чтобы найти y. Умножаем обе части первого уравнения на $-a_2 \neq 0$, а второго – на $a_1 \neq 0$:

$$\begin{cases} -a_1 a_2 x - a_2 b_1 y = -a_2 c_1, \\ a_1 a_2 x + a_1 b_2 y = -a_1 c_2. \end{cases}$$

- Складываем левые и правые части уравнений:

$$(a_1 b_2 - a_2 b_1)y = a_1 c_2 - a_2 c_1$$

$$y = \frac{a_1 c_2 - a_2 c_1}{a_1 b_2 - a_2 b_1}$$

Метод сравнения

- Чтобы решить систему (2) предполагаем, что эта система имеет решение, и из каждого уравнения системы находим y (или x):

$$y = \frac{c_1 - a_1 x}{b_1}, y = \frac{c_2 - a_2 x}{b_2}, b_1 b_2 \neq 0$$

- Приравнивая полученные для y выражения, получаем уравнение относительно неизвестного x. Это уравнение называется выводным.
- Если $a_1 b_2 - a_2 b_1 \neq 0$, то:

$$x = \frac{c_1 b_2 - c_2 b_1}{a_1 b_2 - a_2 b_1}$$

- Подставляя найденное значение x в какое-либо выражение для y (например, первое), получаем:

$$y = \frac{c_1 - a_1 \dfrac{c_1 b_2 - c_2 b_1}{a_1 b_2 - a_2 b_1}}{b_1} = \frac{a_1 c_2 - a_2 c_1}{a_1 b_2 - a_2 b_1}$$

Основные методы решения СЛАУ

- Прямые методы
  - Метод Гаусса
  - Метод Гаусса с выбором главного элемента
  - Метод Холецкого
- Итерационные методы
  - Метод простых итераций
  - Метод Гаусса-Зейделя

Прямые методы

Метод Гаусса

Метод Гаусса с выбором главного элемента

Основные методы решения СЛАУ

- Как представить систему линейных алгебраических уравнений в виде матрицы?

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 = b_1, \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 = b_2, \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 = b_3, \end{cases}$$

$$A = \begin{vmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{vmatrix}$$

$$B = \begin{vmatrix} b_1 \\ b_2 \\ b_3 \end{vmatrix}$$

Метод Гаусса (Gauss Elimination)

- Сводится к нахождению треугольной матрицы;
- Состоит из прямого и обратного хода;
- Необходимое и достаточное условие: неравенство нулю ведущих элементов.

Метод Гаусса (Gauss Elimination)

- Покажем на системе с 4 неизвестными при $a_{11} \neq 0$:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + a_{14}x_4 = b_1, \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + a_{24}x_4 = b_2, \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 + a_{34}x_4 = b_3, \\ a_{41}x_1 + a_{42}x_2 + a_{43}x_3 + a_{44}x_4 = b_4. \end{cases} \quad (3)$$

- Разделим элементы системы (3) на ведущий элемент a_{11}, тогда:

$$x_1 + \frac{a_{12}}{a_{11}}x_2 + \frac{a_{13}}{a_{11}}x_3 + \frac{a_{14}}{a_{11}}x_4 = \frac{b_1}{a_{11}} \quad (4)$$

- Тогда можем исключить неизвестную x_1 из системы (3). Чтобы сделать это, мы должны из второго уравнения системы вычесть уравнение (4) умноженное на a_{21} из третьего уравнения вычесть уравнение (3) умноженное на a_{31} и т.д. Получим:

$$\begin{cases} a_{22}^{(1)}x_2 + a_{23}^{(1)}x_3 + a_{24}^{(1)}x_4 = b_2^{(1)}, \\ a_{32}^{(1)}x_2 + a_{33}^{(1)}x_3 + a_{34}^{(1)}x_4 = b_3^{(1)}, \\ a_{42}^{(1)}x_2 + a_{43}^{(1)}x_3 + a_{44}^{(1)}x_4 = b_4^{(1)}. \end{cases} \quad (5)$$

Где коэффициенты $a_{ij}^{(1)}(i,j \geq 2)$ рассчитываются $a_{ij}^{(1)} = a_{ij} - a_{i1}\frac{a_{1j}}{a_{11}}\ (i,j \geq 2)$.

Метод Гаусса (Gauss Elimination)

- Далее: делим уравнения системы (5) на ведущий элемент $a_{22}^{(1)}$:

$$x_2 + \frac{a_{23}^{(1)}}{a_{22}^{(1)}} x_3 + \frac{a_{24}^{(1)}}{a_{22}^{(1)}} x_4 = \frac{b_2^{(1)}}{a_{22}^{(1)}} \quad (6)$$

- Исключаем x_2 из системы таким же образом и получаем:

$$\begin{cases} a_{33}^{(2)} x_3 + a_{34}^{(2)} x_4 = b_3^{(2)} \\ a_{43}^{(2)} x_3 + a_{44}^{(2)} x_4 = b_4^{(2)} \end{cases} \quad (7)$$

где $a_{ij}^{(2)} = a_{ij}^{(1)} - a_{i2}^{(1)} \frac{a_{2j}^{(1)}}{a_{22}^{(1)}}$

- Делим коэффициенты системы (7) на ведущий элемент $a_{33}^{(2)}$:

$$x_3 + \frac{a_{34}^{(2)}}{a_{33}^{(2)}} x_4 = \frac{b_3^{(2)}}{a_{33}^{(2)}}$$

- Исключаем аналогично x_3:

$$a_{44}^{(3)} x_4 = b_4^{(3)}$$

Где $a_{ij}^{(3)} = a_{ij}^{(2)} - a_{i3}^{(2)} \frac{a_{3j}^{(2)}}{a_{33}^{(2)}}$. Тогда:

$$x_4 = \frac{b_4^{(3)}}{a_{44}^{(3)}}$$

Метод Гаусса (Gauss Elimination)

- Остальные неизвестные последовательно выводятся обратной подстановкой:

$$x_3 = \frac{b_3^{(2)}}{a_{33}^{(2)}} - \frac{a_{34}^{(2)}}{a_{33}^{(2)}} x_4$$

$$x_2 = \frac{b_2^{(1)}}{a_{22}^{(1)}} - \frac{a_{24}^{(1)}}{a_{22}^{(1)}} x_4 - \frac{a_{23}^{(1)}}{a_{22}^{(1)}} x_3$$

$$x_1 = \frac{b_1}{a_{11}} - \frac{a_{14}}{a_{11}} x_4 - \frac{a_{13}}{a_{11}} x_3 - \frac{a_{12}}{a_{11}} x_2.$$

Метод Гаусса (Gauss Elimination)

ДОСТОИНСТВА

- Может быть применен в любом случае, если у системы есть решение и на главной диагонали нет 0 элементов.
- Нет дополнительных подготовительных вычислений.
- Результаты могут быть получены за конечное количество операций.
- Низкая погрешность метода, если производить расчёты в последний момент (например, при вычислении в ручную).

НЕДОСТАТКИ

- При вычислении на компьютере, образуется значительная набегающая погрешность за счёт ограниченности разрядной сетки, особенно при работе с разреженными матрицами (коэффициенты близки к 0).
- Ошибки накапливаются, т.к. расчёт производится последовательно.
- Может занимать существенный объем памяти для больших матриц, так как всю матрицу нужно держать в памяти.
- Алгоритмическая сложность метода $O(n^3)$, где n – число неизвестных.

Метод Гаусса с выбором главного элемента (Gauss Elimination through pivoting)

- Требование $a_{ii} \neq 0$ заменяется более жестким: из всех оставшихся в **i**-м столбце элементов нужно выбрать наибольший по модулю и переставить уравнения так, чтобы этот элемент оказался на месте элементаa_{ii};
- Существует три вариации метода:
  - Искать максимальный элемент в **столбце** и переставлять **строки** так, чтобы максимальный элемент оказался ведущим элементом;
  - Искать максимальный элемент в **строке** (уравнении) и переставлять **столбцы**;
  - Искать максимальный элемент **во всей матрице** и производить перестановки и **строк, и столбцов**.

Метод Гаусса с выбором главного элемента (Gauss Elimination through pivoting)

- Выберем ненулевой, как правило, наибольший по модулю, не принадлежащий к столбцу свободных членов $a_{pq}, (q \neq n+1)$ из матрицы M, и вычислим множители:

$$m_i = -\frac{a_{iq}}{a_{pq}}$$

для всех $i \neq p$

- Строка с номером p матрицы M , содержащая главный элемент, называется главной строкой.
- Далее, произведем следующую операцию: к каждой не главной строке прибавим главную строку, умноженную на соответствующий множитель **mi** для этой строки. В результате мы получим новую матрицу, у которой **q**-й столбец состоит из нулей. Отбрасывая этот столбец и главную **p**-ю строку, получим новую матрицу **M** (1) с меньшим на единицу числом строк и столбцов.
- Это стратегия перестановки строк.

Метод Гаусса с выбором главного элемента (Gauss Elimination through pivoting)

ДОСТОИНСТВА

- Найдёт решение, если оно существует.
- **Лучше работает с разреженными матрицами, потому что делим на максимальный элемент, а не на любой диагональный.**
- Решение найдется за конечное число операций.
- Ниже погрешность метода, если выполнять операции в последний момент, например при вычислении в ручную.

НЕДОСТАТКИ

- При вычислении на компьютере, образуется значительная набегающая погрешность за счёт ограниченности разрядной сетки, особенно при работе с разреженными матрицами (коэффициенты близки к 0). **Однако, ошибка ниже, чем у метода Гаусса.**
- Ошибки накапливаются, т.к. расчёт производится последовательно.
- Может занимать существенный объем памяти для больших матриц, так как всю матрицу нужно держать в памяти.
- Алгоритмическая сложность метода $O(n^3)$, где n – число неизвестных.

Вычисление определителя

- Оба прямых метода основаны на исключении во время прямого хода прямом и замене при обратном ходе. Они имеют треугольную матрицу после прямого исключения.
- После всех таких операций мы можем вычислить определитель исходной матрицы по треугольной матрице, поскольку использовались только элементарные преобразования.
- Чтобы найти определитель треугольной матрицы, вы можете просто умножить элементы только с главной диагонали.

Невязка (Residual error)

- Очень полезно вычислять остаточные ошибки (невязки) после использования прямых методов. Это помогает понять меру ошибки ваших расчетных значений неизвестного.
- Невязка - это разница между левой и правой частями уравнения. Теоретически оно должно быть равно 0 (по определению уравнения, т.е. равенства).
- Но на практике, после большого количества операций, таких как деление, например, в случае ограниченного компьютерного представления чисел и с приблизительными числами в результате, результирующий вектор неизвестного может иметь значительную разницу.
- Чтобы вычислить невязку, мы помещаем вычисленное неизвестное исходной системе в каждое уравнение, а затем вычитаем одну сторону из другой.

$$\begin{cases} a_1 x + b_1 y = c_1, \\ a_2 x + b_2 y = c_2, \end{cases} \Rightarrow \begin{cases} |a_1 x + b_1 y - c_1| = r_1, \\ |a_2 x + b_2 y - c_2| = r_2, \end{cases}$$

- Таким образом, у нас будет вектор остаточных ошибок для каждого уравнения (не для каждого неизвестного!)

Cholesky (Khaletsky) decomposition (Cholesky factorization)

Разложение Холецкого

Cholesky (Khaletsky) decomposition (Cholesky factorization)

- Применение:
  - Решение СЛАУ
  - Аппроксимация наименьшими квадратами
  - Решение ОДУ (задачи Коши)
  - Методы Монте-Карло для случайных выборок

Cholesky (Khaletsky) decomposition (Cholesky factorization)

- Пусть система линейных алгебраических уравнений имеет вид: $Ax = b$, где $A = [a_{ij}]$ - квадратная матрица размера n и:

$$x = \begin{bmatrix} x_1 \\ \dots \\ x_n \end{bmatrix}, b = \begin{bmatrix} a_{1,n+1} \\ \dots \\ a_{n,n+1} \end{bmatrix}$$

- Представим матрицу $\mathbf{A}$ в виде произведения нижнедиагональной матрицы $B = [b_{ij}]$ и верхнедиагональной матрицы $C = [c_{ij}]$, где на главной диагонали будут содержаться 1; тогда: $A = BC$, где:

$$B = \begin{bmatrix} b_{11} & 0 & \cdots & 0 \\ b_{21} & b_{22} & \cdots & 0 \\ \cdots & \cdots & \cdots & \cdots \\ b_{n1} & b_{n2} & \cdots & b_{nn} \end{bmatrix}, C = \begin{bmatrix} 1 & c_{12} & \cdots & c_{1n} \\ 0 & 1 & \cdots & c_{2n} \\ \cdots & \cdots & \cdots & \cdots \\ 0 & 0 & \cdots & 1 \end{bmatrix}$$

- Тогда элементы b_{ij}, c_{ij} определяются по формулам:

$$\begin{cases} b_{i1} = a_{i1}, \\ b_{ij} = a_{ij} - \sum_{k=1}^{j-1} b_{ik} c_{kj} \ (i \geq j > 1) \end{cases}, \text{ and } \begin{cases} c_{1j} = \frac{a_{1j}}{b_{11}}, \\ c_{ij} = \frac{1}{b_{ii}} \left(a_{ij} - \sum_{k=1}^{i-1} b_{ik} c_{kj} \right) (1 < i < j) \end{cases}$$

Cholesky (Khaletsky) decomposition (Cholesky factorization)

- Откуда мы желаемый вектор x может быть вычислен из цепочки уравнений: $By = b, Cx = y$
- Эта система легко решается, поскольку матрицы B и C являются треугольными:

$$\begin{cases} y_1 = \dfrac{a_{1,n+1}}{b_{11}}, \\ y_i = \dfrac{1}{b_{ii}}\left(a_{i,n+1} - \displaystyle\sum_{k=1}^{i-1} b_{ik} y_k\right) (i > 1) \end{cases} \quad \text{and} \quad \begin{cases} x_n = y_n, \\ x_i = y_i - \displaystyle\sum_{k=i+1}^{n} c_{ik} x_k \ (i < n) \end{cases}$$

- Числа y_i преимущественно вычисляются вместе с коэффициентами c_{ij}.
- Эта схема использует обычный тип проверки с помощью сумм.
- Если матрица A симметричная, такая что $a_{ij} = a_{ji}$, тогда $c_{ij} = \frac{b_{ij}}{b_{ii}}, (i < j)$

Cholesky (Khaletsky) decomposition (Cholesky factorization)

- Пример:

$$\begin{cases} 3x_1 + x_2 - x_3 + 2x_4 = 6; \\ -5x_1 + x_2 + 3x_3 - 4x_4 = -12; \\ 2x_1 + x_3 - x_4 = 1; \\ x_1 - 5x_2 + 3x_3 - 3x_4 = 3. \end{cases}$$

	x_1	x_2	x_3	x_4		$\sum$	x_1	x_2	x_3	x_4		$\sum$
I	a_{11}	a_{12}	a_{13}	a_{14}	a_{15}	a_{16}	3	1	-1	2	6	11
	a_{21}	a_{22}	a_{23}	a_{24}	a_{25}	a_{26}	-5	1	3	-4	-12	-17
	a_{31}	a_{32}	a_{33}	a_{34}	a_{35}	a_{36}	2	0	1	-1	1	3
	a_{41}	a_{42}	a_{43}	a_{44}	a_{45}	a_{46}	1	-5	3	-3	3	-1
II	b_{11}	1	c_{12}	c_{13}	c_{14}	c_{15}	c_{16}	3	1	0,333..	−0,333..	0,667..	2	3,6667..
	b_{21}	b_{22}	1	c_{23}	c_{24}	c_{25}	c_{26}	-5	2,67..	1	0,5	-0,25	-0,75	0,5
	b_{31}	b_{32}	b_{33}	1	c_{34}	c_{35}	c_{36}	2	−0,6667..	2	1	-1,25	-1,25	-2
	b_{41}	b_{42}	b_{43}	b_{44}	1	c_{45}	c_{46}	1	−5,3333..	6	2,5	1	3	4
III					y_1	x_1					2	1
					y_2	x_2					-0,75	-1
					y_3	x_3					-1,25	2
					y_4	x_4					3	3

Итерационные методы

Метод простых итераций

Метод Гаусса-Зейделя

Метод простых итераций

$$x_i^{[j+1]} = x_i^{[j]} - \frac{1}{a_{ii}} f_i^{[j]} = x_i^{[j]} - \frac{1}{a_{ii}} \left(\sum_{k=1}^{n} a_{ik} x_k^{[j]} - b_i \right)$$

- Существует множество стратегий выбора начальных значений$x_i^{[j]}$ для выполнения первой итерации:
  - 0
  - b_i
  - некоторое предварительно рассчитанное значение (например, из прямых методов) для повышения точности результата
  - любое другое значение
- Итерационный процесс сходится, если:

$$|a_{ii}| \geq \sum_{i \neq k} |a_{ik}| \ (i, k = 1, 2, \dots, n)$$

- Условие окончания итерационного процесса за конечное число операций:

$$\forall i (i = 1,2, \dots, n) \ \forall \varepsilon > 0 : \left| x_i^{[j]} - x_i^{[j-1]} \right| \leq \varepsilon$$

Метод простых итераций

ДОСТОИНСТВА

- Может использоваться, когда вы хотите минимизировать погрешность результатов (с помощью настройки ε по мере необходимости).
- Нет необходимости хранить полную матрицу в памяти.
- Ошибки не накапливаются как для прямых методов.
- Вычисление каждого уравнения текущей итерации независимо друг от друга (только для предыдущей итерации), и поэтому может быть распараллелено (рассчитано одновременно).

НЕДОСТАТКИ

- На практике для больших матриц редко найти такую, которая бы удовлетворяла условию сходимости.
- Все неизвестные рассчитываются на основе значений, полученных на предыдущей итерации. Если вычислять уравнения последовательно, то уже известны новые значения неизвестных для частей уравнений.
- Если обозначить число итераций за l, тогда алгоритмическая сложность метода $O(l * n^2)$. Вы не знаете как много итераций потребуется (скорость сходимости) – l может быть как больше, так и меньше n.

Метод Гаусса-Зейделя

$$x_i^{[j+1]} = x_i^{[j]} - \frac{1}{a_{ii}}\left(\sum_{k=1}^{i-1} a_{ik}x_k^{[j+1]} + \sum_{k=i+1}^{n} a_{ik}x_k^{[j]} - b_i\right)$$

- Правила выбора начальных значений неизвестного и условия завершения итерационного процесса такие же, как и для метода простых итераций.
- Итерационный процесс будет сходиться, если:

$$|a_{ii}| \geq \sum_{i \neq k} |a_{ik}| \ (i, k = 1, 2, \dots, n)$$

NB! Но хотя бы для одного уравнения должно выполняться условие:

$$|a_{ii}| > \sum_{i \neq k} |a_{ik}| \ (i, k = 1, 2, \dots, n)$$

Метод Гаусса-Зейделя

ДОСТОИНСТВА

- Может использоваться, когда вы хотите минимизировать погрешность результатов (с помощью настройки ε по мере необходимости).
- Нет необходимости хранить полную матрицу в памяти.
- Ошибки не накапливаются как для прямых методов.
- Скорость сходимости может быть выше, чем для метода простых итераций, т.к. используются свежеполученные значения неизвестных с текущей итераций.

НЕДОСТАТКИ

- Сложнее параллелизовать – все уравнения зависят друг от друга в пределах текущей итерации.
- Условие сходимости более строгое, чем у метода простых итераций. Сложнее найти систему, для которой можно было бы применить метод.
- Если обозначить число итераций за l, тогда алгоритмическая сложность метода $O(l * n^2)$. Вы не знаете как много итераций потребуется (скорость сходимости) – l может быть как больше, так и меньше n.

Алгоритмическая сложность метода, Big O Notation

- 1. Если алгоритм выполняет определенную последовательность шагов f(N) раз для математической функции f, он выполняет O(f(N)) шагов.
- 2. Если алгоритм выполняет операцию, которая занимает O(f(N)) шагов, а затем выполняет вторую операцию, которая занимает O(g(N)) шагов для функций f и g, общая производительность алгоритма равна O(f(N) + g(N)).
- 3. Если алгоритм принимает O(f (N) + g (N)), а функция f (N) больше, чем g (N) для больших N, производительность алгоритма может быть упрощена до O(f(N)).
- 4. Если алгоритм выполняет операцию, которая занимает O(f (N)) шагов, и для каждого шага в этой операции он выполняет еще O (g (N)) шагов, общая производительность алгоритма равна O(f(N) × g(N)).
- 5. Игнорируйте постоянные кратные числа. Если C является константой, O(C × f(N)) совпадает с O(f(N)), а O(f(C × N)) совпадает с O(f(N)).

Спасибо за внимание!

В случае вопросов по лекции задавайте их через форму:

https://forms.yandex.ru/u/61ffab0425b437e0e3410e9b/

Мы обязательно обсудим их на следующем занятии.